



Programming & Software Engineering Practices

A redesigned technical reference that preserves the source material while presenting it in a new documentation template.

FUNDAMENTALS	Structured learning reference
PRACTICAL CODE	Structured learning reference
ENGINEERING PRACTICES	Structured learning reference
REFERENCE	Structured learning reference

Contents

Part A — Python Language and Libraries

- 1. Indentation and Comments
- 2. Functions
- 3. Object-Oriented Programming (OOP)
- 4. File Handling
- 5. Exception Handling
- 6. NumPy and Pandas
- 7. Web Scraping with Selenium
- 8. Regular Expressions (Regex)
- 9. Multithreading and Multiprocessing
- 10. Concurrency vs. Parallelism

Part B — Software Engineering Practices

- 11. Software Development Life Cycle (SDLC)
- 12. Agile and Scrum
- 13. Code Version Control
- 14. Documentation Practices

- 15. Risk Management
- 16. Python Coding Standards and PEP 8
- 17. Comments and Docstrings
- 18. Error Handling and Logging
- 19. Writing Efficient Code
- 20. Software Design Principles
- 21. Unit Testing and Validation
- 22. Code Quality Tools: Ruff and Black
- 23. Summary

1

Indentation and Comments

1.1 Indentation

- Python uses indentation (whitespace at the start of a line) to define blocks of code, instead of braces { } used in many other languages.
- Consistent indentation is not just a style choice — it is mandatory. Mixing tabs and spaces, or using inconsistent spacing, raises an `IndentationError`.
- The widely accepted standard is 4 spaces per indentation level.

```
if True:
```

```
    print("This line is indented")
```

```
if True:
```

```
    print("This line is nested further")
```

1.2 Comments

- Single-line comments start with a `#` and continue to the end of the line.
- There is no true multi-line comment syntax; developers typically use consecutive `#` lines, or a triple-quoted string as a block comment.
- Comments should explain why code does something, not simply restate what it does.

```
# This is a single-line comment
```

```
x = 10 # Set the initial count
```

```
"""
```

```
This is a multi-line block,
```

```
often used as a temporary comment
```

```
or module-level description.
```

```
"""
```

2

Functions

2.1 What Is a Function?

- A function is a reusable, named block of code defined using the `def` keyword.
- Functions help organize code, avoid repetition, and make programs easier to test and maintain.

Page 3 of 20

3

Object-Oriented Programming (OOP)**3.1 Core Concepts**

Concept

Description

Class

A blueprint for creating objects, defining their attributes and behavior

Object

A specific instance created from a class

Page 4 of 20

Python — Programming & Software Engineering Practices

Concept

Description

Encapsulation

Bundling data and methods together, restricting direct access to internal state

Inheritance

A class acquiring properties and methods from another (parent) class

Polymorphism

The same method name behaving differently depending on the object calling it

Abstraction

Hiding complex implementation details behind a simple interface

3.2 A Simple Class

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return f"{self.name} makes a sound."
class Dog(Animal):
    def speak(self):
        return f"{self.name} barks."
d = Dog("Rex")
print(d.speak()) # Rex barks.
```

3.3 Types of Methods

Method Type

Description

Instance method

Operates on a specific object; takes self as its first parameter

Class method (@classmethod)

Operates on the class itself; takes cls as its first parameter

Static method (@staticmethod)

A regular function grouped inside a class; does not access the instance or class

Page 5 of 20

4.1 Opening Files

Mode

Description

r

Read (default) — file must already exist

w

Write — creates a new file or overwrites an existing one

a

Append — adds new content to the end of an existing file

r+

Read and write, without truncating the file

rb / wb

Read/write in binary mode

4.2 Reading and Writing

```
with open("notes.txt", "w") as f:
```

```
    f.write("Hello, file!")
```

```
with open("notes.txt", "r") as f:
```

```
    content = f.read()
```

```
print(content)
```

4.3 The with Statement

Using `with open(...)` as `f` automatically closes the file once the block finishes, even if an error occurs. This is the recommended way to handle files, avoiding resource leaks that can happen if a file is opened manually and never explicitly closed.

4.4 Working with CSV and JSON

```
import csv, json
```

```
with open("data.csv") as f:
```

```
    reader = csv.reader(f)
```

```
    for row in reader:
```

```
        print(row)
```

```
with open("data.json") as f:
```

```
    data = json.load(f)
```

Page 6 of 20

5

Exception Handling

5.1 try / except / else / finally

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
else:
    print("Runs only if no exception occurred")
finally:
    print("Always runs, error or not")
```

5.2 Common Built-in Exceptions

Exception

Raised When

ValueError

A function receives an argument of the correct type but an invalid value

TypeError

An operation is applied to an object of an inappropriate type

KeyError

A dictionary key is not found

IndexError

A sequence index is out of range

FileNotFoundError

A file or directory that does not exist is accessed

5.3 Raising Custom Exceptions

```
class InsufficientFundsError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError("Not enough balance")
    return balance - amount
```

6

NumPy and Pandas

6.1 NumPy

NumPy provides fast, memory-efficient arrays and vectorized mathematical operations, forming the foundation of most Python data science and numerical computing libraries.

Page 7 of 20

7

Web Scraping with Selenium

7.1 What Is Selenium?

- Selenium is a browser automation tool that can control a real web browser programmatically.
- Unlike simple HTML-parsing libraries, Selenium can interact with JavaScript-heavy pages, click buttons, fill forms, and wait for content to load.

Page 8 of 20

8

Regular Expressions (Regex)

8.1 What Is Regex?

A regular expression is a pattern used to match, search, or manipulate text. Python's built-in `re` module provides regex support.

8.2 Common Metacharacters

Symbol

Meaning

.

Matches any single character except a newline

Page 9 of 20

9

Multithreading and Multiprocessing**9.1 The Global Interpreter Lock (GIL)**

CPython's Global Interpreter Lock allows only one thread to execute Python bytecode at a time. This limits true parallel execution of CPU-bound Python code across threads, even on multi-core machines.

Page 10 of 20

10

Concurrency vs. Parallelism

Concept

Description

Concurrency

Multiple tasks make progress by interleaving, without necessarily running at the exact same instant — useful for I/O-bound work

Parallelism

Multiple tasks run literally at the same time on separate CPU cores — useful for CPU-bound work

- Concurrency is about dealing with many things at once; parallelism is about doing many things at once.
- Python threads provide concurrency, but not true parallelism for CPU-bound code, due to the GIL.
- Python's multiprocessing module provides genuine parallelism by using separate processes, each with its own interpreter and memory space.

Page 11 of 20

11 Software Development Life Cycle (SDLC)

The SDLC is a structured process for planning, creating, testing, and deploying software.

Phase

Description

Requirement Analysis

Gathering and documenting what the software needs to do

Design

Planning the system architecture and technical approach

Implementation

Writing the actual code

Testing

Verifying the software works as intended and is free of defects

Deployment

Releasing the software into a live environment

Maintenance

Fixing issues and adding improvements after release

11.1 Common SDLC Models

- Waterfall — a strict, sequential model where each phase completes before the next begins.
- Iterative — the system is built in repeated cycles, refining it each time.
- Spiral — combines iterative development with structured risk analysis at each cycle.
- Agile — delivers working software in small, frequent increments, adapting to change quickly.

12 Agile and Scrum

12.1 Agile Principles

- Deliver working software frequently, in short cycles.
- Welcome changing requirements, even late in development.
- Favor close, daily collaboration between business and technical teams.
- Value individuals and interactions over rigid processes and tools.

12.2 Scrum Roles

Role

Responsibility

Product Owner

Defines and prioritizes what needs to be built

Page 12 of 20

13

Code Version Control

13.1 Why Version Control?

- Tracks every change to code over time, with a full history of who changed what and when.
- Enables multiple developers to work on the same codebase without overwriting each other's work.
- Makes it possible to safely experiment, branch, and roll back changes if something breaks.

13.2 Common Git Commands

Command

Purpose

`git init`

Initialize a new Git repository

`git clone url`

Copy an existing remote repository locally

`git add .`

Stage changes for the next commit

`git commit -m "message"`

Save staged changes with a descriptive message

`git push`

Upload local commits to a remote repository

`git pull`

Fetch and merge changes from a remote repository

Page 13 of 20

14 Documentation Practices

14.1 Why Documentation Matters

- Helps new team members understand a codebase faster.
- Preserves knowledge that would otherwise be lost when team members change.
- Reduces repeated questions and miscommunication across a team.

14.2 Common Types of Documentation

Type

Purpose

README

A quick overview of a project, how to install and run it

API Documentation

Describes available endpoints, functions, and their parameters

Inline Comments / Docstrings

Explain specific pieces of code directly where they live

Architecture Documentation

Describes how components of a system fit together

Popular documentation-generation tools for Python projects include Sphinx, which builds documentation from docstrings and reStructuredText, and MkDocs, which builds simple documentation sites from Markdown files.

15 Risk Management

Risk management is the process of identifying, assessing, and responding to potential problems before they impact a software project.

Page 14 of 20

16 Python Coding Standards and PEP 8

PEP 8 is Python's official style guide, describing conventions for writing clear, consistent, and readable code.

Guideline

Recommendation

Indentation

Use 4 spaces per indentation level; never mix tabs and spaces

Line Length

Limit lines to a maximum of 79-99 characters

Naming — variables/functions

snake_case (e.g. calculate_total)

Naming — classes

PascalCase (e.g. CustomerOrder)

Naming — constants

UPPER_SNAKE_CASE (e.g. MAX_RETRIES)

Imports

One import per line, grouped as standard library, third-party, then

local

Whitespace

Avoid extraneous whitespace inside parentheses and before commas

17 Comments and Docstrings

17.1 Writing Good Comments

- Explain the reasoning behind non-obvious code, not what the code trivially does.
- Keep comments up to date — an outdated comment is often worse than no comment at all.

17.2 Docstrings

A docstring is a string literal placed as the first statement in a module, function, class, or method, describing its purpose. Docstrings are accessible at runtime via the `__doc__` attribute and used by documentation tools.

Page 15 of 20

18

Error Handling and Logging

18.1 Best Practices for Exception Handling

- Catch specific exceptions rather than using a bare `except:`, which can hide real bugs.
- Only catch exceptions you can meaningfully handle or report.
- Use `finally` or context managers to guarantee cleanup (closing files, releasing connections).

18.2 The logging Module

Python's built-in logging module is generally preferred over `print()` for tracking events in an application, since it supports severity levels, timestamps, and configurable output destinations.

Level

Typical Use

DEBUG

Detailed diagnostic information, useful during development

INFO

Confirmation that things are working as expected

WARNING

Something unexpected happened, but the program continues

ERROR

A more serious problem prevented a function from completing

CRITICAL

A severe error that may prevent the program from continuing

Page 16 of 20

19

Writing Efficient Code

- Use list comprehensions instead of manual loops for simple transformations — they are often faster and more readable.
- Use generators (yield) for large datasets to avoid loading everything into memory at once.
- Prefer built-in functions and libraries (sum(), sorted(), NumPy) — they are implemented in optimized C code.
- Cache expensive function results with functools.lru_cache when the same inputs repeat.
- Avoid unnecessary work inside loops, such as repeated attribute lookups or re-computation of constant values.

```
from functools import lru_cache
```

```
# List comprehension
```

```
squares = [x * x for x in range(10)]
```

```
# Generator (memory-efficient)
```

```
def read_large_file(path):
```

```
    with open(path) as f:
```

```
        for line in f:
```

```
            yield line.strip()
```

```
# Caching expensive calls
```

```
@lru_cache(maxsize=None)
```

```
def fibonacci(n):
```

```
    if n < 2:
```

```
        return n
```

```
    return fibonacci(n - 1) + fibonacci(n - 2)
```

```
Page 17 of 20
```

20

Software Design Principles

20.1 SOLID Principles

Principle

Idea

Single Responsibility

A class or function should have only one reason to change

Open/Closed

Software entities should be open for extension, but closed for modification

Liskov Substitution

Subclasses should be usable in place of their parent class without breaking behavior

Interface Segregation

Prefer several small, specific interfaces over one large, general one

Dependency Inversion

Depend on abstractions, not concrete implementations

20.2 Other Common Principles

Principle

Meaning

DRY (Don't Repeat Yourself)

Avoid duplicating logic; extract it into a single, reusable place

KISS (Keep It Simple, Stupid)

Prefer the simplest solution that solves the problem correctly

YAGNI (You Aren't Gonna Need

It)

21

Unit Testing and Validation

21.1 What Is Unit Testing?

- Unit tests verify that a small, isolated piece of code (typically a single function) behaves correctly.
- Automated tests catch regressions early, before they reach production.

21.2 Testing Frameworks

Framework

Description

unittest

Python's built-in testing framework, based on xUnit-style test

classes

Page 18 of 20

Tool

Purpose

Black

An opinionated code formatter that automatically rewrites code into a consistent style

Ruff

An extremely fast linter (and formatter) that checks code for errors, style issues, and PEP 8 violations

Format all files in the project with Black

\$ black .

Lint the project with Ruff

\$ ruff check .

Automatically fix lint issues where possible

\$ ruff check . --fix

Why Use Both?

Black focuses purely on consistent formatting (spacing, line breaks, quote style), while Ruff focuses on catching bugs, unused imports, and style violations — together they keep a codebase clean with minimal manual effort.

Page 19 of 20

This documentation has covered the Python language from its fundamentals — indentation, comments, functions, and object-oriented programming — through practical skills like file and exception handling, data analysis with NumPy and Pandas, web scraping with Selenium, pattern matching with regex, and concurrent execution using multithreading and multiprocessing. It then moved beyond the language itself into the software engineering practices that shape professional projects: the SDLC, Agile and Scrum, version control, documentation, risk management, coding standards, testing, and the tools — Ruff and Black — that help enforce them automatically.

Key Takeaway

Writing good Python code is only half the job — following disciplined engineering practices around testing, version control, documentation, and code quality tooling is what turns working code into a maintainable, professional software product.

Page 20 of 20